



Laravel Eloquent Relationships: An Advanced Guide

Ship and manage your web projects faster

Deploy your projects on Google Cloud Platform's top tier infrastructure. You'll get 25+ data centers to choose from, 24/7/365 expert support, and advanced security with DDoS protection.

Try for free

■ Data centers (25+) ■ CDN locations (200)

Often there is a point in every developer's life where you have to interact with a database. Here is where Eloquent, Laravel's object-relational mapper (ORM), makes the process of interacting with your database tables intuitive and natural.

It is vital that as a professional, you should recognize and understand the six key relationship types which we will go through and review.

What Are Relationships in Eloquent?

When working with tables in a relational database, we can characterize relationships as connections between tables. This helps you organize and structure data effortlessly allowing for superior readability and handling of data. There are three types of database relationships in practice:

- one-to-one – One record in a table is associated with one, and only one, in another table. For example, a person and a social security number.
- one-to-many – One record is associated with multiple records in another table. For instance, a writer and their blogs.

- many-to-many – Multiple records in a table are associated with multiple records in another table. Namely, students and the courses they are enrolled in.

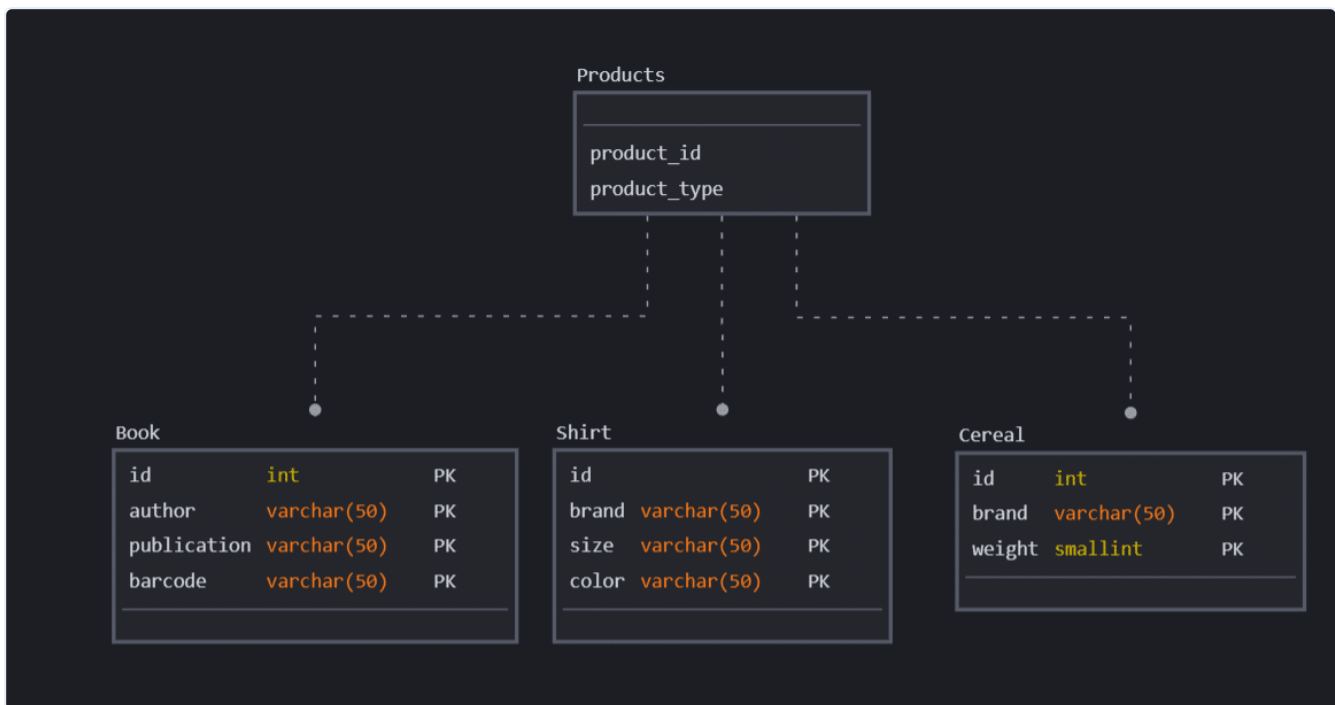
Laravel makes it seamless to interact and manage database relationships using object-oriented syntax in Eloquent.

Along with these definitions, Laravel introduces more relationships, namely:

- Has Many Through
- Polymorphic Relations
- Many-to-many Polymorphic

Take, for example, a store whose inventory contains a variety of articles, each in its own category. Therefore, splitting the database into multiple tables makes sense from a business point of view. This comes with issues of its own, as you do not want to query each and every single table.

We can easily create a simple one-to-many relation in Laravel to help us out, such as when we need to query the products, we can do it by using the Product model.



— Database schema with three tables and a joint table representing a polymorphic relationship

One-To-One Relationship

Being the first basic relation Laravel offers, they associate two tables in a way such that one row from the first table is correlated with only one row from the other table.

To see this in action, we have to create two models with their own migration:

```
php artisan make:model Tenant
php artisan make:model Rent
```

At this point, we have two models, one being the Tenant and the other being their Rent.

```

<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Tenant extends Model
{
    /**
     * Get the rent of a Tenant
     */
    public function rent()
    {
        return $this->hasOne(Rent::class);
    }
}

```

Because eloquent determines the foreign key relationship based on the parent model name (Tenant in this case), the Rent model assumes that there exists a **tenant_id** foreign key.

We can easily overwrite it like with an additional argument to the **hasOne** method:

```

return $this->hasOne(Rent::class, "custom_key");

```

Eloquent also assumes that there is a match between the defined foreign key and the primary key of the parent (Tenant model). By default, it will look to match **tenant_id** with the **id** key of the Tenant record. We can overwrite this with a third argument in the **hasOne** method, such that it will match another key:

```
return $this->hasOne(Rent::class, "custom_key", "other_key");
```

Now that we have defined the one-to-one relationship between the models, we can use it easily, like this:

```
$rent = Tenant::find(10)->rent;
```

With this line of code, we get the tenant's rent with the id 10 if it exists.

One-To-Many Relationship

Like the previous relationship, this will define relationships between a single-parent model and multiple children models. It is unlikely that our Tenant will have only one Rent bill because it is a recurring payment, therefore, he will have multiple payments.

In this case, our previous relationship has flaws, and we can fix them:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Tenant extends Model
{
```

```
/**
 * Get the rents of a Tenant
 */
public function rent()
{
    return $this->hasMany(Rent::class);
}
}
```

Before we call the method to get the rents, a good thing to know is that relationships serve as query builders, so we can further add constraints (like rent in between dates, min payment, etc.) and chain them to get our desired result:

```
$rents = Tenant::find(10)->rent()->where('payment', '>', 500)->first();
```

And like the previous relationship, we can overwrite the foreign and local keys by passing additional arguments:

```
return $this->hasMany(Rent::class, "foreign_key");
```

```
return $this->hasMany(Rent::class, "foreign_key", "local_key");
```

Now we have all the rent of a tenant, but what do we do when we know the rent and want to figure out to whom it belongs? We can make use of the **belongsTo** property:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Rent extends Model
{
    /**
     * Return the tenant for the rent
     */
    public function tenant()
    {
        return $this->belongsTo(Tenant::class);
    }
}
```

And now we can get the tenant easily:

```
$tenant = Rent::find(1)->tenant;
```

For the **belongsTo** method, we can also overwrite the foreign and local keys as we did before.

Has-One-Of-Many Relationship

Since our Tenant model can be associated with many Rent models, we want to easily retrieve the latest or oldest related model of the relationships.

A convenient way of doing this is combining the **hasOne** and **ofMany** methods:

```
public function latestRent() {
    return $this->hasOne(Rent::class)->latestOfMany();
}

public function oldestRent() {
    return $this->hasOne(Rent::class)->oldestOfMany();
}
```

By default, we are getting the data based on the primary key, which is sortable, but we can create our own filters for the **ofMany** method:

```
return $this->hasOne(Rent::class)->ofMany('price', 'min');
```

HasOneThrough and HasManyThrough Relationships

The **-Through** methods suggest that our models will have to go through another one other model to establish a relationship with the wanted model. For example, we can associate the Rent with the Landlord, but the Rent must first go through the Tenant to reach the Landlord.

The keys of the tables necessary for this would look like this:

```
rent
  id - integer
  name - string
  value - double

tenants
  id - integer
  name - string
  rent_id - integer

landlord
  id - integer
  name - string
  tenant_id - integer
```

After visualizing how our tables look, we can make the models:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Rent extends Model
{
    /**
     * Return the rents' landlord
     */
    public function rentLandlord()
    {
        return $this->hasOneThrough(Landlord::class, Tenant::class);
    }
}
```

```
}
```

The first argument of the **hasOneThrough** method is the model you want to access, and the second argument is the model you will go through.

And just like before, you can overwrite the foreign and local keys. Now that we have two models, we have two of each to overwrite in this order:

```
public function rentLandlord()
{
    return $this->hasOneThrough(
        Landlord::class,
        Tenant::class,
        "rent_id",    // Foreign key on the tenant table
        "tenant_id",  // Foreign key on the landlord table
        "id",         // Local key on the tenant class
        "id"          // Local key on the tenant table
    );
}
```

Similarly, the “Has Many Through” relationship in Laravel Eloquent is useful when you want to access records in a distant table through an intermediate table. Let’s consider an example with three tables:

- country
- users
- games

Each Country has many Users, and each User has many Games. We want to retrieve all Games belonging to a Country through the User table.

You would define the tables like this:

```
country
  id - integer
  name - string

user
  id - integer
  country_id - integer
  name - string

games
  id - integer
  user_id - integer
  title - string
```

Now you should define the Eloquent model for each and every table:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Country extends Model
{
    protected $fillable = ['name'];

    public function users()
    {
        return $this->hasMany(User::class);
    }
}
```

```
    }

    public function games()
    {
        return $this->hasManyThrough(Games::class, User::class);
    }
}
```

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class User extends Model
{
    protected $fillable = [article_id, 'name'];

    public function country()
    {
        return $this->belongsTo(Country::class);
    }

    public function posts()
    {
        return $this->hasMany(Post::class);
    }
}
```

```
<?php
```

```

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Game extends Model
{
    protected $fillable = ['user_id', 'title'];

    public function user()
    {
        return $this->belongsTo(User::class);
    }
}

```

Now we can call the **games()** method of the Country model to get all the games because we established the “Has Many Through” relationship between Country and Game through the User model.

```

<?php

$country = Country::find(159);

// Retrieve all games for the country
$games = $country->games;

```

Many-To-Many Relationship

The many-to-many relationship is more complicated. One good example is an employee that has multiple roles. A role can also be assigned to multiple employees. This is the basis of the many-to-many relationship.

For this, we must have the **employees**, **roles**, and **role_employees** tables.

Our database table structure will look like this:

```
employees
  id - integer
  name - string

roles
  id - integer
  name - string

role_employees
  user_id - integer
  role_id - integer
```

Knowing the relationship's tables structure, we can easily define our **Employee model** to **belongsToMany Role** model.

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Employee extends Model
{
    public function roles()
    {
        return $this->belongsToMany(Role::class);
    }
}
```

Once we defined this, we can access all the roles of an employee and even filter them:

```
$employee = Employee::find(1);
$employee->roles->forEach(function($role) { // });

// OR

$employee = Employee::find(1)->roles()->orderBy('name')->where('name', 'a
```

Like all other methods, we can overwrite the foreign and local keys of the **belongsToMany** method.

To define the inverse relationship of the **belongsToMany** we simply use the same method but on the child method now, with the parent as an argument.

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Role extends Model
{
    public function employees()
    {
        return $this->belongsToMany(Employee::class);
    }
}
```

Uses of The Intermediate Table

As we may have noticed, when we use the many-to-many relationship, we are always supposed to have an intermediate table. In this case, we are using the **role_employees** table.

By default, our pivot table will contain only the id attributes. If we want other attributes, we have to specify them like so:

```
return $this->belongsToMany(Employee::class)->withPivot("active", "created_at");
```

If we want to shortcut the pivot for the timestamps, we can do:

```
return $this->belongsToMany(Employee::class)->withTimestamps();
```

One trick to know is that we can customize the 'pivot' name into anything that suits our application better:

```
return $this->belongsToMany(Employee::class)->as('subscription')->withPivot('active', 'created_at');
```

Filtering the results of an eloquent query is a must-know for any developer that wants to step up their game and [optimize their Laravel applications](#).

Therefore Laravel provides a fantastic feature of pivots where that can be used to filter the data we want to collect. So instead of using other features like [database transactions](#) to get our data in chunks, we can filter it with useful methods like **wherePivot**, **wherePivotIn**, **wherePivotNotIn**, **wherePivotBetween**, **wherePivotNotBetween**, **wherePivotNull**, **wherePivotNotNull** and we can use them when defining relationships between tables!

```
return $this->belongsToMany(Employee::class)->wherePivot('promoted', 1);
return $this->belongsToMany(Employee::class)->wherePivotIn('level', [1, 2]);
return $this->belongsToMany(Employee::class)->wherePivotNotIn('level', [1, 2]);
return $this->belongsToMany(Employee::class)->wherePivotBetween('posted_at', 1, 2);
return $this->belongsToMany(Employee::class)->wherePivotNull('expired_at');
return $this->belongsToMany(Employee::class)->wherePivotNotNull('posted_at');
```

One last amazing feature is that we can order by pivots:

```
return $this->belongsToMany(Employee::class)
    ->where('promoted', true)
    ->orderByPivot('hired_at', 'desc');
```

Polymorphic Relationships

The word Polymorphic comes from Greek, and it means “many forms.” Like this, one model in our application can take many forms, meaning it can have more than one association. Imagine we are building an application with blogs, videos, polls, etc. A user can create a comment for any of these. Therefore, a **Comment model** might belong to **Blogs**, **Videos**, and **Polls** models.

Polymorphic One To One

This type of relationship is similar to a standard one-to-one relationship. The only difference is that the child model can belong to more than one type of model with a single association.

Take, for example, a **Tenant** and **Landlord** model, it may share a polymorphic relation to a **WaterBill** model.

The table structure can be as follows:

```
tenants
  id - integer
  name - string

landlords
  id - integer
  name - string

waterbills
  id - integer
  amount - double
  waterbillable_id
  waterbillable_type
```

We are using `waterbillable_id` for the id of the **landlord** or **tenant**, while the `waterbillable_type` contains the class name of the parent model. The type column is used by eloquent to figure out what parent model to return.

The model definition for such a relationship will look as follows:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class WaterBill extends Model
{
    public function billable()
    {
        return $this->morphTo();
    }
}

class Tenant extends Model
{
    public function waterBill()
    {
        return $this->morphOne(WaterBill::class, 'billable');
    }
}

class Landlord extends Model
{
    public function waterBill()
    {
        return $this->morphOne(WaterBill::class, 'billable');
    }
}
```

Once we have all of this in place, we can access the data from both the Landlord and Tenant model:

```
<?php
```

```
$tenant = Tenant::find(1)->waterBill;  
$landlord = Landlord::find(1)->waterBill;
```

Polymorphic One To Many

This is similar to a regular one-to-many relation, the only key difference is that the child model can belong to more than one type of a model, using a single association.

In an application like Facebook, users can comment on posts, videos, polls, live, etc. With a polymorphic one to many, we can use a single **comments** table to store the comments for all the categories we have. Our tables structure would look something like this:

```
posts  
  id - integer  
  title - string  
  body - text  
  
videos  
  id - integer  
  title - string  
  url - string  
  
polls  
  id - integer  
  title - string  
  
comments  
  id - integer
```

```
body - text
commentable_id - integer
commentable_type - string
```

The commentable_id being the id of the record, and the commentable_type being the class type, so eloquent knows what to look for. As for the model structure, it is very similar to the polymorphic one to one:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Comment extends Model
{
    public function commentable()
    {
        return $this->morphTo();
    }
}

class Poll extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}

class Live extends Model
{
    public function comments()
    {
```

```
        return $this->morphMany(Comments::class, 'commentable');  
    }  
}
```

Now to retrieve the comments of a Live, we can simply call the find method with the id, and now we have access to the comments iterable class:

```
<?php  
  
use App\Models\Live;  
  
$live = Live::find(1);  
  
foreach ($live->comments as $comment) { }  
  
// OR  
  
Live::find(1)->comments()->each(function($comment) { // });  
Live::find(1)->comments()->map(function($comment) { // });  
Live::find(1)->comments()->filter(function($comment) { // });  
  
// etc.
```

And if we have the comment and want to find out to whom it belongs, we access the commentable method:

```
<?php

use App\Models\Comment;

$comment = Comment::find(10);
$commentable = $comment->commentable;

// commentable - type of Post, Video, Poll, Live
```

Polymorphic One of Many

In a lot of applications that scale, we want an easy way to interact with models and between them. We may want a user's first or last post, which can be done with a combination of **morphOne** and **ofMany** methods:

```
<?php

public function latestPost()
{
    return $this->morphOne(Post::class, 'postable')->latestOfMany();
}

public function oldestPost()
{
    return $this->morphOne(Post::class, 'postable')->oldestOfMany();
}
```

The methods **latestOfMany** and **oldestOfMany** are retrieving the latest or oldest model based on the model's primary key, which was the condition that it is sortable.

In some cases, we do not want to sort by the ID, maybe we changed the publishing date of some posts and we want them in that order, not by their id.

This can be done by passing 2 parameters to the **ofMany** method to help with this. The first parameter is the **key** that we want to filter by, and the second is the **sorting method**:

```
<?php

public function latestPublishedPost()
{
    return $this->morphOne(Post::class, "postable")->ofMany("published_at")
}
```

With this in mind, it is possible to construct more advanced relations for this! Imagine we have this scenario. We are asked to generate a list of all current posts in the order they have been published. The problem arises when we have 2 posts with the same published_at value and when posts are scheduled to be posted in the future.

To do this, we can pass the order in which we want the filters to be applied to the **ofMany** method. This way we order by published_at, and if they are the same, we order by id. Secondly, we can apply a query function to the **ofMany** method to exclude all posts that are scheduled for publishing!

```
<?php

public function currentPosts()
{
    return $this->hasOne(Post::class)->ofMany([
        'published_at' => 'max',
    ])
}
```

```
        'id' => 'max',
    ], function ($query) {
        $query->where('published_at', '<', now());
    });
}
```

Polymorphic Many To Many

The polymorphic many-to-many is slightly more complex than the normal one. One common situation is having tags apply to more assets in your application. For example, on TikTok, we have tags that can be applied to Videos, Shorts, Stories, etc.

The polymorphic many-to-many allows us to have a single table of tags associated with the Videos, Shorts, and Stories.

The table structure is simple:

```
videos
  id - integer
  description - string

stories
  id - integer
  description - string

taggables
  tag_id - integer
  taggable_id - integer
  taggable_type - string
```

With the tables ready, we can make the model and use the **morphToMany** method. This method accepts the name of the model class and the 'relationship name':

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Video extends Model
{
    public function tags()
    {
        return $this->morphToMany(Tag::class, 'taggable');
    }
}
```

And with this, we can easily define the inverse relation. We know that for every child model we want to call the **morphedByMany** method:

```
<?php

namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Tag extends Model
{
    public function stories()
    {
        return $this->morphedByMany(Story::class, 'taggable');
    }
}
```

```
public function videos()
{
    return $this->morphedByMany(Video::class, 'taggable');
}
```

And now, when we get a Tag, we can retrieve all videos and stories related to that tag!

```
<?php
use App\Model\Tag;

$tag = Tag::find(10);
$posts = $tag->stories;
$videos = $tag->stories;
```

Optimize Eloquent for Speed

When working with Laravel's Eloquent ORM, it's essential to understand how to optimize database queries and minimize the amount of time and memory it requires to fetch data. One way to do this is by [implementing caching](#) in your application.

Laravel provides a flexible caching system that supports various backends, such as [Redis](#), [Memcached](#), and [file-based caching](#). By caching Eloquent query results, you can reduce the number of database queries, making your application faster and more valuable.

Additionally, you can use Laravel's [query builder](#) to create additional complex queries, further optimizing your application's performance.

Summary

In conclusion, Eloquent relationships are a powerful feature of Laravel that allows developers to easily work with related data. From one-to-one to many-to-many relationships, Eloquent provides a simple and intuitive syntax to define and query these relationships.

As a Laravel developer, mastering Eloquent relationships can greatly enhance your development workflow and make your code more efficient and readable. If you're interested in learning more about Laravel, Kinsta has various resources available, including a [tutorial](#) on getting started with Laravel and an article on [Laravel developer salaries](#).

Kinsta offers cloud hosting solutions that make [deploying and managing Laravel applications](#) a breeze.